Practical 1

Creation of Test Suite Using Selenium IDE

AIM:

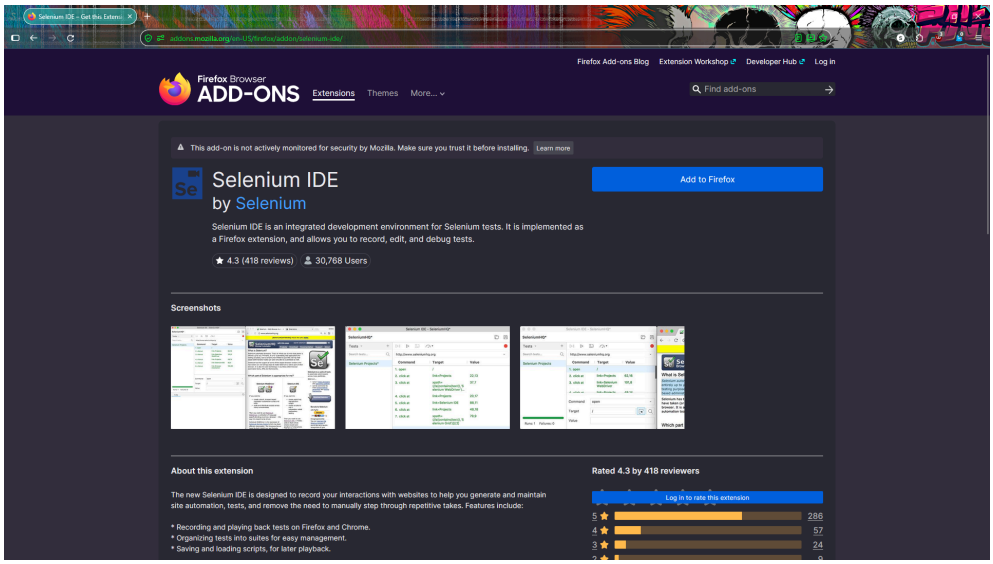
Install Selenium IDE and create a test suite containing a minimum of four test cases to validate different web page formats (HTML page, XML response, JSON API output, and a dynamic form page). Execute the suite and export the automation script in WebDriver forma

REQUIREMENTS:

1. Selenium IDE extension on a browser.
2. NetBeans IDE for creating test case files.

PROCEDURE:

1. Search for “Selenium IDE Chrome” and install the Selenium IDE browser extension.



2. Open NetBeans IDE and create a Web Application:
   File > New Project > Java Web > Web Application > Add GlassFish Server > Finish.
3. Create index.html with the following code.

--- index.html ---

```
<html>
<head>
   <title>prac1</title>
   <script language="javascript">
```

```
        function addition() {
            var num1 = parseInt(document.arithmetic.n1.value);
            var num2 = parseInt(document.arithmetic.n2.value);
            var result = num1 + num2;
            document.arithmetic.res.value = result;
        }

        function subtraction() {
            var num1 = parseInt(document.arithmetic.n1.value);
            var num2 = parseInt(document.arithmetic.n2.value);
            var result = num1 - num2;
            document.arithmetic.res.value = result;
        }

        function multiplication() {
            var num1 = parseInt(document.arithmetic.n1.value);
            var num2 = parseInt(document.arithmetic.n2.value);
            var result = num1 * num2;
            document.arithmetic.res.value = result;
        }

        function division() {
            var num1 = parseInt(document.arithmetic.n1.value);
            var num2 = parseInt(document.arithmetic.n2.value);
            var result = num1 / num2;
            document.arithmetic.res.value = result;
        }
    </script>
</head>

<body>
    <h1 align="center">Arithmetic Operations</h1>
    <form name="arithmetic">
        <table border="1" align="center">
            <tr>
                <td>Number 1:</td>
                <td><input type="text" name="n1" size="20"></td>
            </tr>
            <tr>
```

```
            <td>Number 2:</td>
            <td><input type="text" name="n2" size="20"></td>
         </tr>
         <tr>
            <td colspan="2">
               <input type="button" name="add" value="Add"
                  onclick="javascript:addition();"> 
               <input type="button" name="sub" value="Subtract"
                  onclick="javascript:subtraction();"> 
               <input type="button" name="mul" value="Multiply"
                  onclick="javascript:multiplication();"> 
               <input type="button" name="div" value="Divide"
                  onclick="javascript:division();"> 
            </td>
         </tr>
         <tr>
            <td colspan="2">Result is:
               <input type="text" name="res" size="20" value="">
            </td>
         </tr>
      </table>
   </form>
</body>
</html>
```

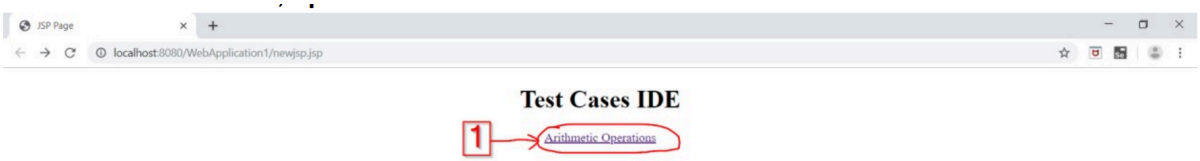
4. Create a JSP file named newjsp.jsp with a link to index.html.

--- newjsp.jsp ---

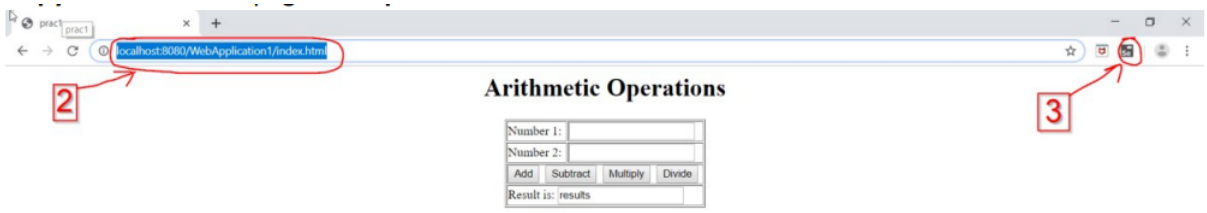
```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
   <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
   <title>JSP Page</title>
</head>
<body align="center">
   <h1>Test Cases IDE</h1>
   <a href="index.html">Arithmetic Operations</a>
```

</body>
</html>

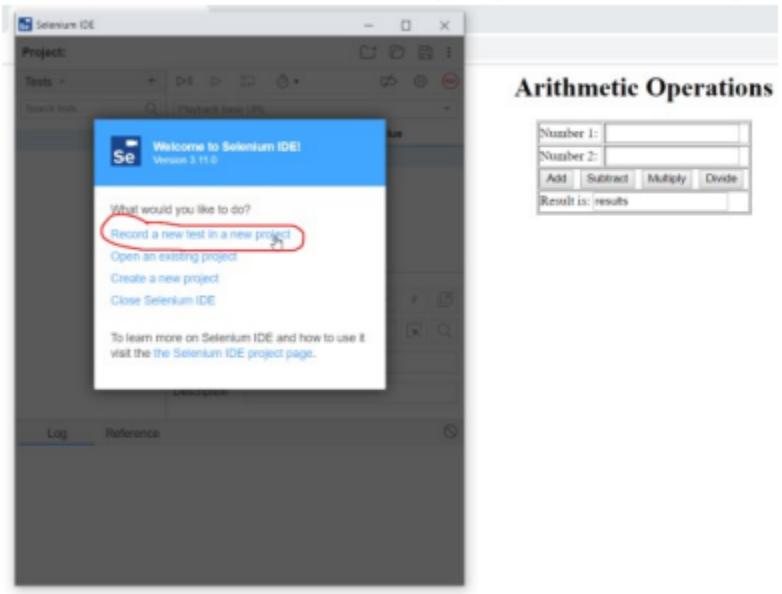
5. Run newjsp.jsp from NetBeans. In the browser, click the “Arithmetic Operations” link.



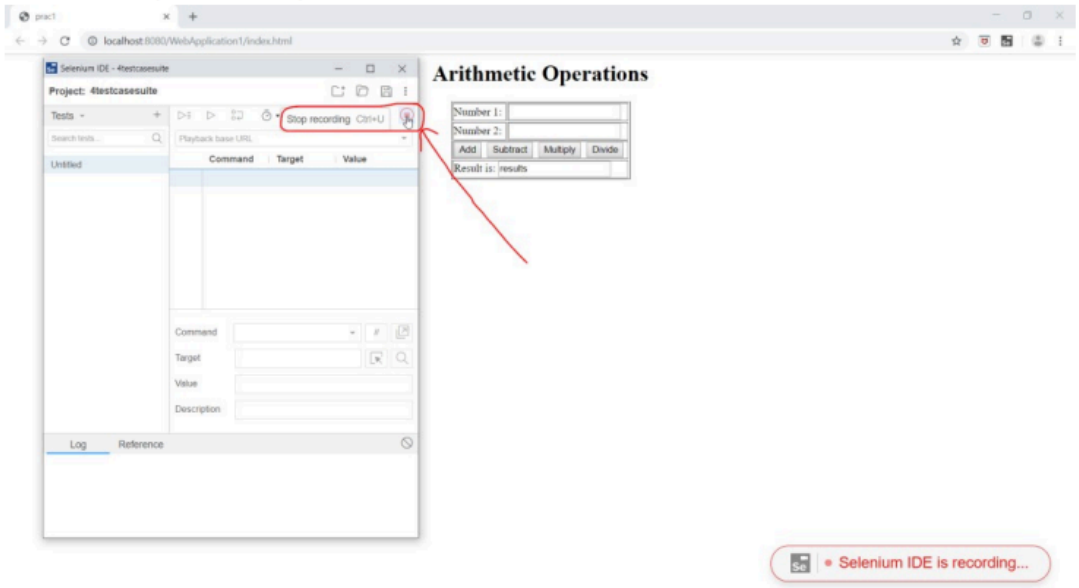
6. Copy the URL of the Arithmetic Operations page and open the Selenium IDE extension.



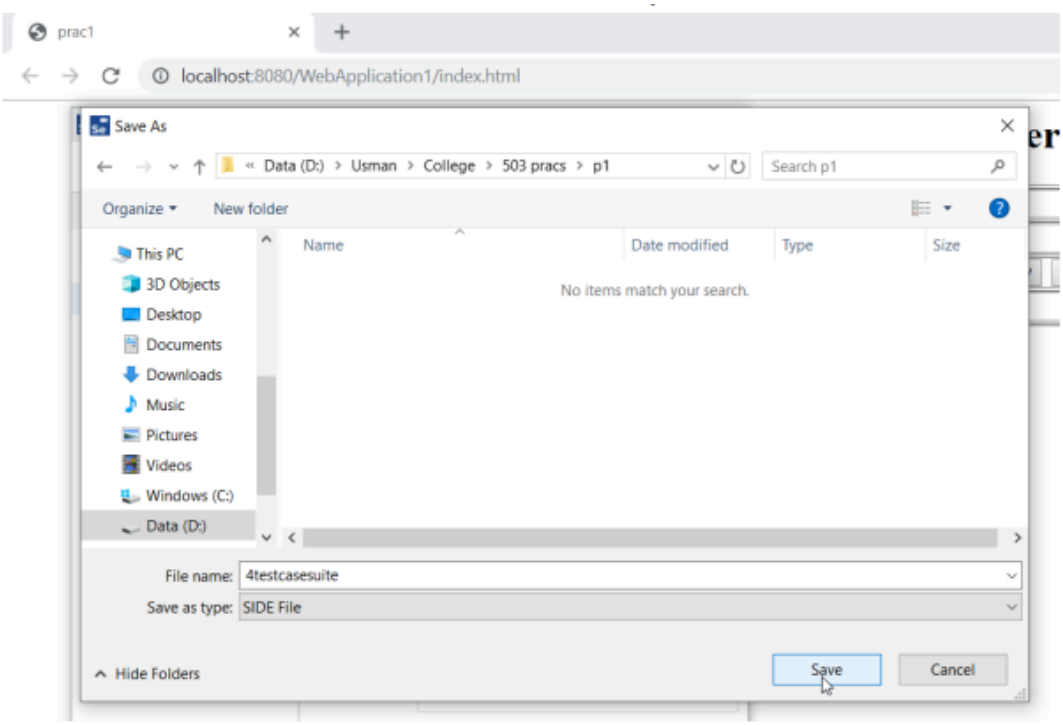
7. Create a new test project, enter a project name, paste the page URL, and click “Start Recording”.



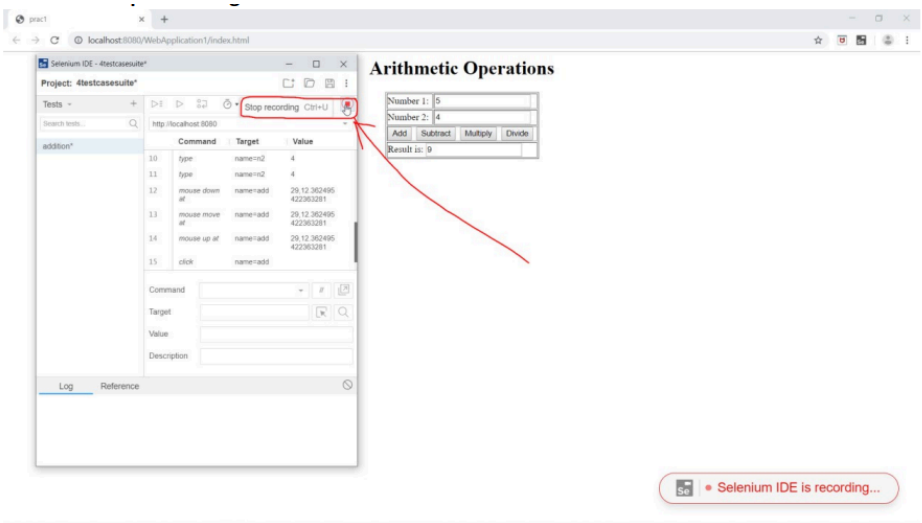
8. A new tab opens. Perform the actions for the first test case and then click “Stop Recording”.



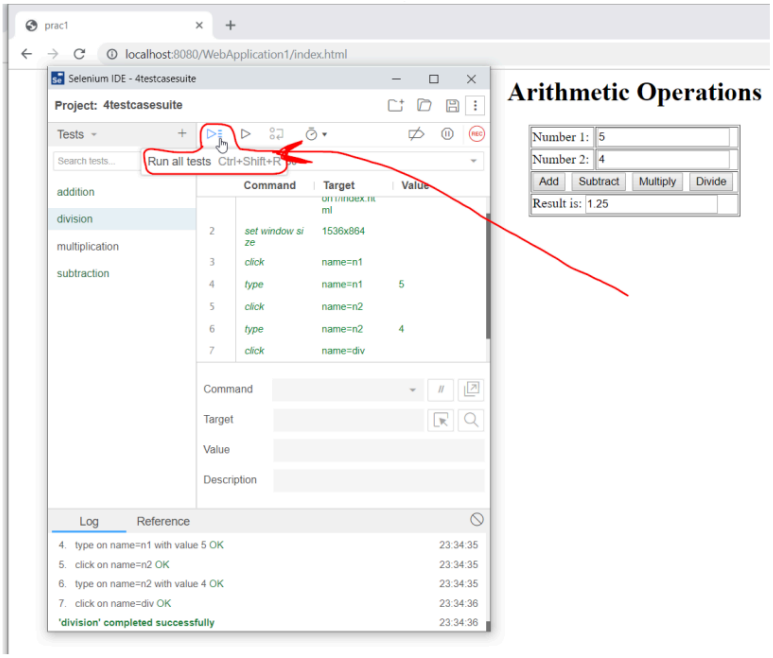
9. Name the test case “addition” and save the project. The project is saved with the .SIDE extension.



10. Start recording again using the same URL. Record the steps for addition, stop recording, and run the current test. Check the execution in the Selenium IDE Log area.



11. Repeat the process to create three more test cases: subtraction, multiplication, and division.
12. Use “Run all tests” to execute the complete test suite.


prac1
localhost:8080/WebApplication1/index.html
Selenium IDE - 4testcasesuite
Project: 4testcasesuite
Tests
Search tests...
Run all tests Ctrl+Shift+R
addition
division
multiplication
subtraction
Command
Target
Value
2 set window size 1536x864
3 click name=n1
4 type name=n1 5
5 click name=n2
6 type name=n2 4
7 click name=div
Command
Target
Value
Description
Log
Reference
4. type on name=n1 with value 5 OK 23:34:35
5. click on name=n2 OK 23:34:35
6. type on name=n2 with value 4 OK 23:34:35
7. click on name=div OK 23:34:36
'division' completed successfully 23:34:36
Arithmetic Operations
Number 1: 5
Number 2: 4
Add
Subtract
Multiply
Divide
Result is: 1.25

Practical 2

Cross-Website Functional Testing Using Selenium IDE

AIM:

Develop and execute a test suite for two different websites. Perform functional operations such as clicking links, filling forms, selecting dropdown options, and verifying displayed content. Analyze test results and generate execution logs.

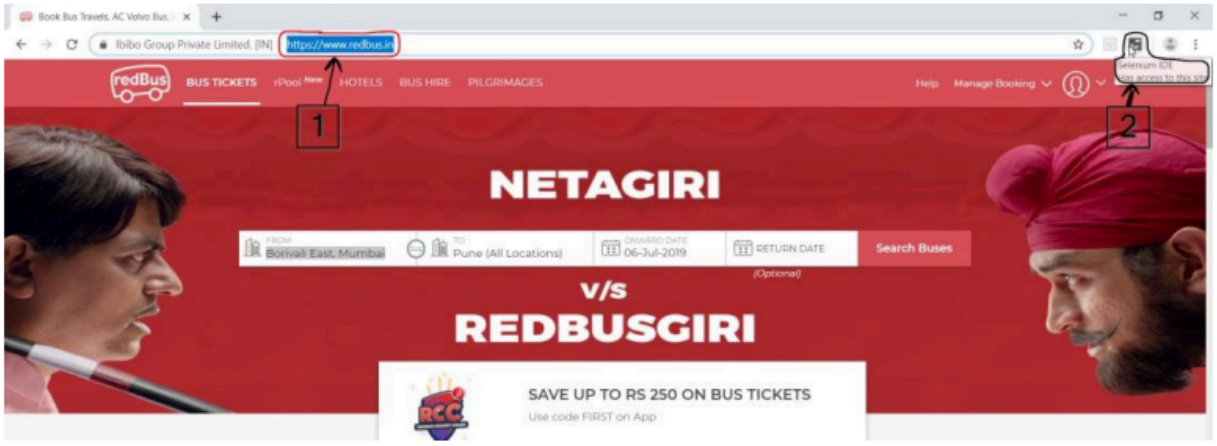
REQUIREMENTS:

1. Selenium IDE extension on a browser.
2. A stable Internet connection.

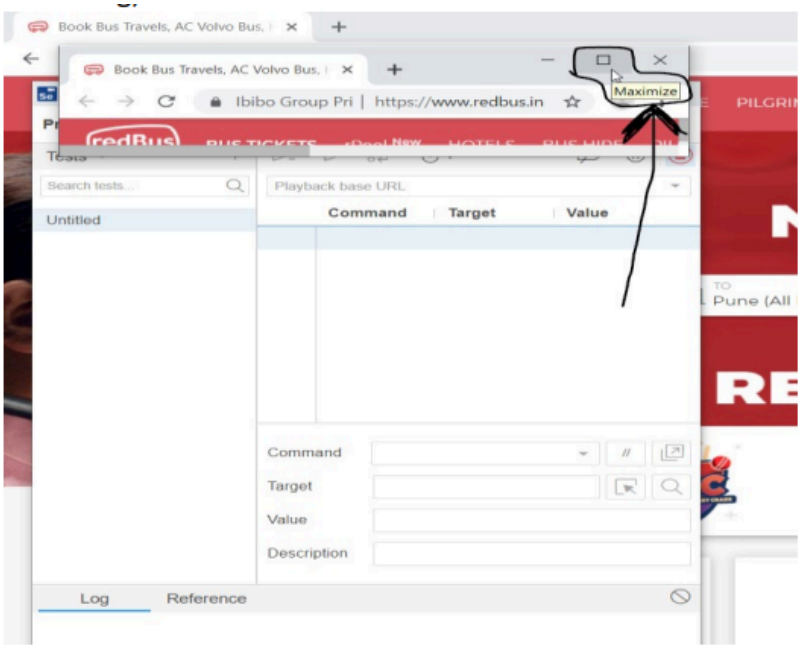
PROCEDURE:

PART A: TESTING REDBUS.IN

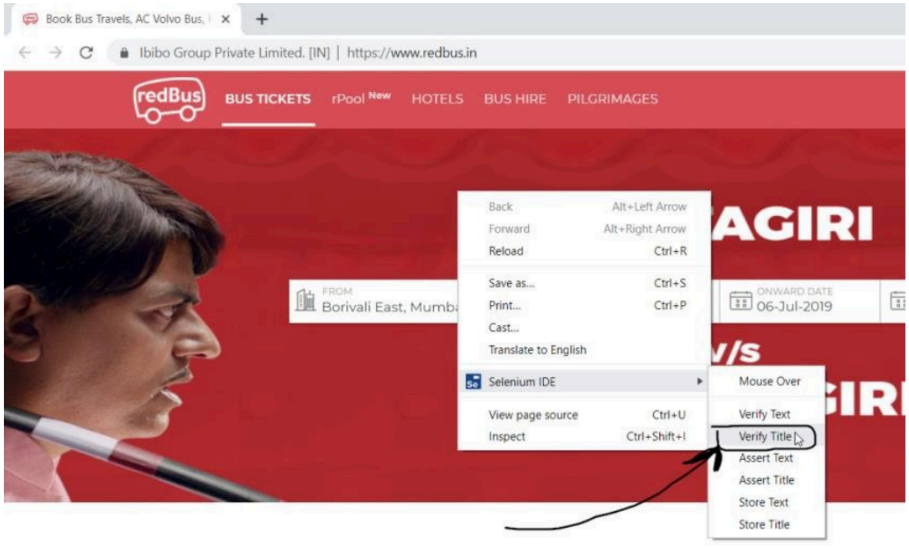
1. Search for “Selenium IDE Chrome” and install the Selenium IDE extension in the browser.
2. Open the RedBus website (redbus.in). Copy its URL and open Selenium IDE.



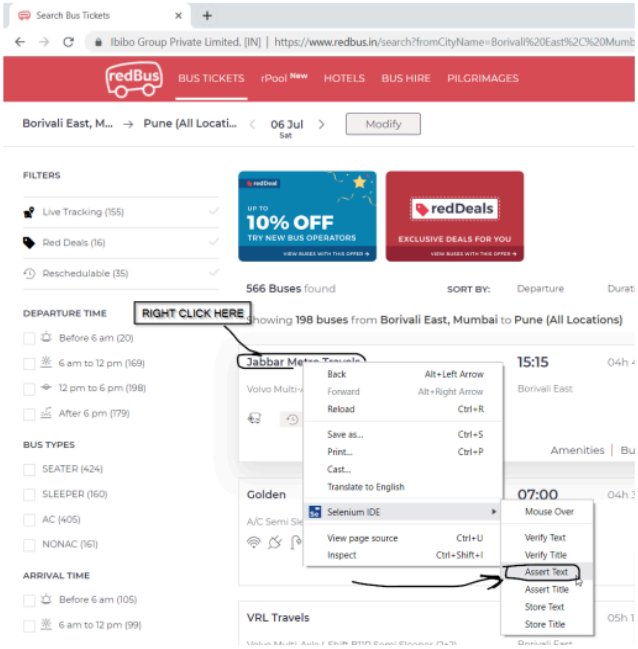
3. Create a new test project, enter a project name, paste the RedBus URL, and click “Start Recording”.
4. A new browser tab will open. Maximize it and use it as the working tab for recording.



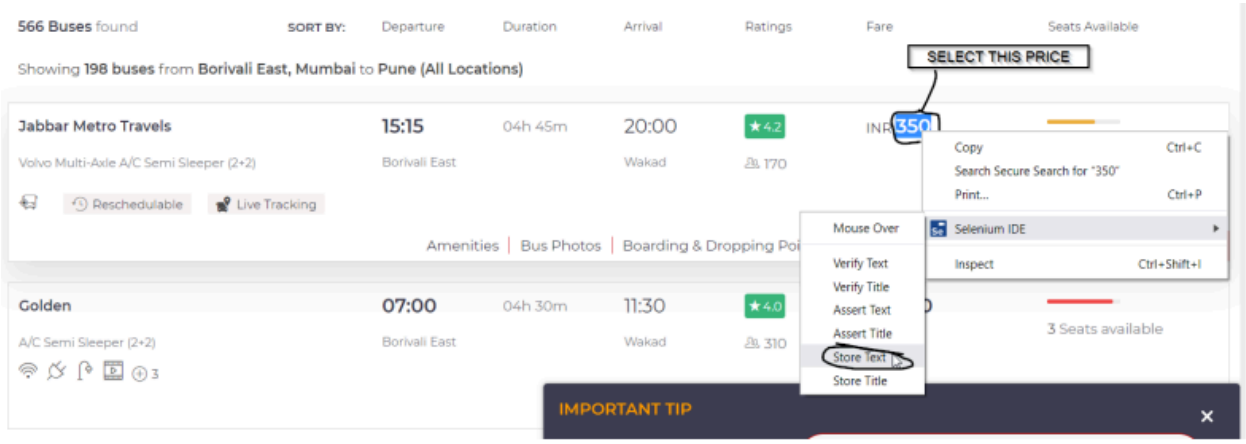
5. Use the Selenium IDE command “verifyTitle”.
   - Right-click anywhere on the page.
   - Select Selenium IDE > Verify Title.
   - This command checks the title of the webpage and continues execution.



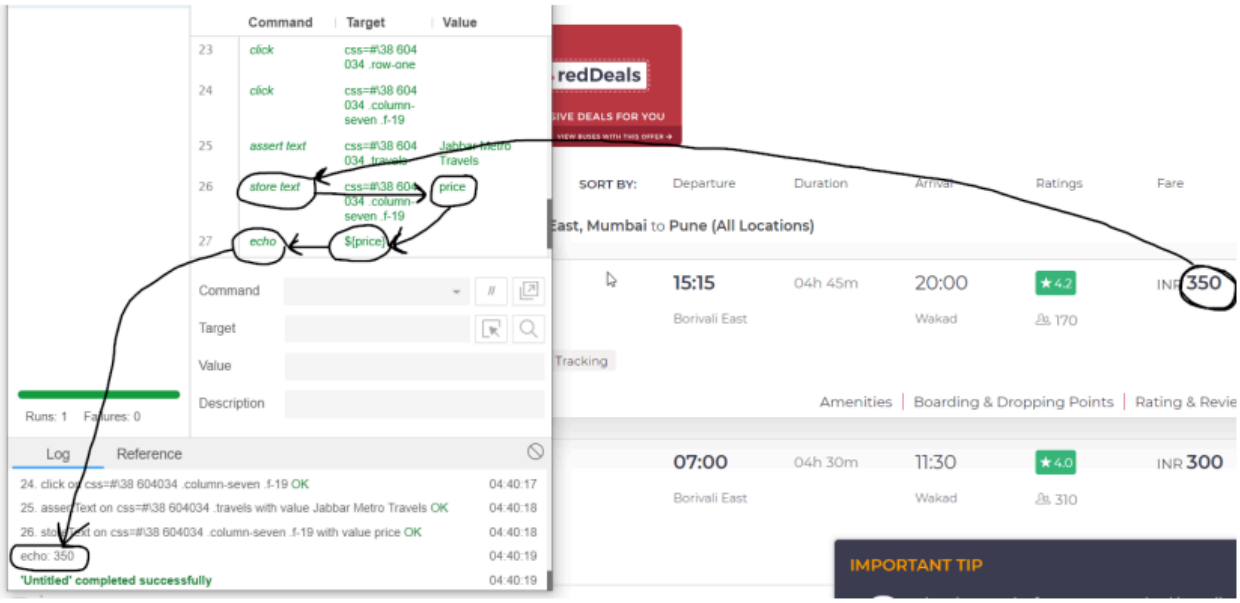
6. Enter the FROM location, TO location, and ONWARD DATE. Click “Search Buses”. Avoid unnecessary clicks while recording.
7. Use the “assertText” command to verify the text of the selected bus provider.
   - Right-click the bus provider name.
   - Select Selenium IDE > Assert Text.
   - If the expected text is not found, the test case stops.



8. Use the “storeText” command to store the bus ticket price.
   - Select the ticket price, right-click it, and choose Selenium IDE > Store Text.
   - Enter “price” as the variable name when prompted.



9. Add an “echo” command on the next line to display the stored price.
   - Command: echo
   - Value: ${price}
10. Run the current test. Check the Log area in Selenium IDE. The journal example displays “350” as the stored price.



Practical 3

Manual Test Case Design and Execution

Aim:

Prepare a Test Plan, Test Scenarios, and Test Cases for a given web application. Execute test cases manually and prepare a Test Execution Report and Defect Report as per STLC process.

Tool Required:

MS Excel / Google Sheets

Objectives:

To understand the manual testing process.
To design effective test cases.
To prepare Test Execution Report and Defect Report.

Theory:

Manual Testing is a software testing process in which test cases are executed manually by a tester without using any automation tools. It helps to verify whether the application works according to the specified requirements.

The Software Testing Life Cycle (STLC) defines different phases involved in testing:

Requirement Analysis:

Understanding the application requirements and identifying the features to be tested.

Test Planning:

Preparing a test plan that defines the testing scope, objectives, resources, and schedule.

Test Case Design:

Creating test scenarios and detailed test cases with test steps, expected results, and test data.

Test Environment Setup:

Preparing the required environment for executing test cases.

Test Execution:

Executing test cases manually and comparing actual results with expected results.

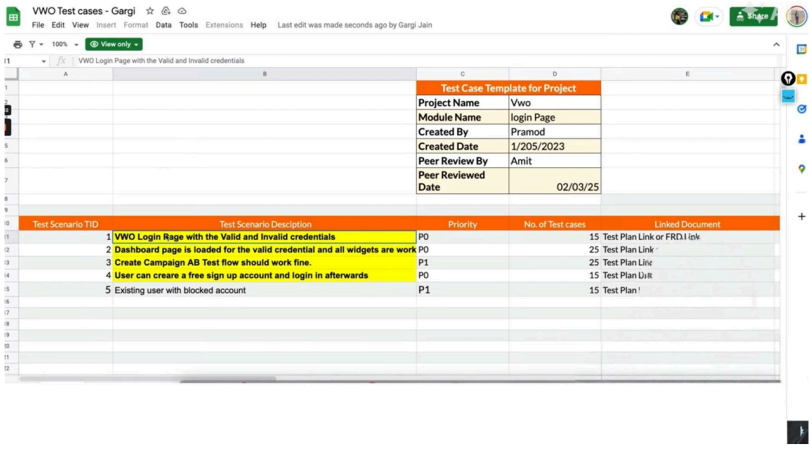
Defect Reporting:

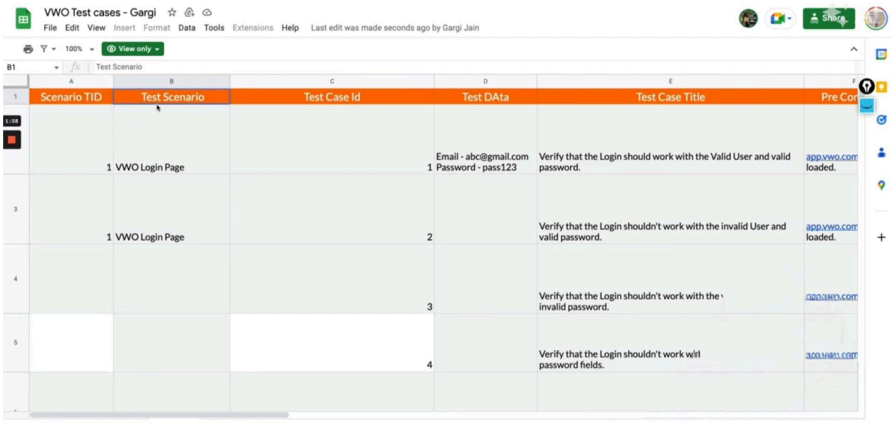
Recording defects found during testing.

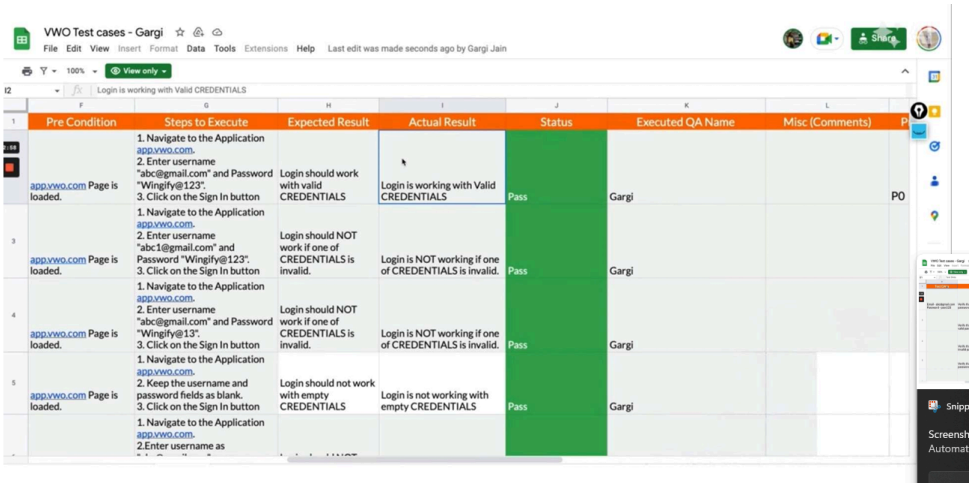
Test Closure:

Preparing the final test execution report after completion of testing.

Expected Output :







Practical 4

Selenium Server (Grid) Configuration and Remote Execution

AIM:

Install and configure Selenium Server in Hub-Node architecture. Execute a test script on multiple browsers using Remote WebDriver and compare cross-browser results.

TOOLS REQUIRED:

1. Java (JDK 8 or above)
2. Selenium Server (Standalone JAR)
3. Selenium WebDriver (Java)
4. Eclipse IDE
5. Google Chrome and Mozilla Firefox

OBJECTIVES:

1. To understand Selenium Grid architecture.
2. To perform cross-browser testing.
3. To execute test scripts remotely using RemoteWebDriver.

PROCEDURE:

Step 1: Install Prerequisites

1. Install Java JDK.
2. Install Eclipse IDE.
3. Download the Selenium Server JAR.
4. Add Selenium libraries to the Java project.

Step 2: Start Selenium Hub

1. Open Command Prompt.
2. Navigate to the folder where the Selenium Server JAR file is stored by executing:

```
cd /d "D:\TYCS STQA\Selenium Server"
```

3. Start the Hub using the following command:

```
java -jar selenium-server-4.23.0.jar hub
```

Step 3: Verify Hub

1. Open a browser.

2. Enter the following URL:

   http://localhost:4444

3. The Selenium Grid page should open.

Step 4: Start Node
1. Open another Command Prompt window.
2. Execute the following command:

   java -jar selenium-server-4.23.0.jar node

3. The Node will automatically connect to the Hub.
4. Refresh the Selenium Grid page. The connected Node will be displayed.

Step 5: Create Java Project
1. Open Eclipse.
2. Create a Java Project.
3. Add Selenium libraries to the project:
   - Right-click the project.
   - Select Build Path > Configure Build Path.
   - Open Libraries and click Add External JARs.
   - Navigate to the D:\TYCS STQA folder.
   - Open the Selenium-java-4.23.0 folder and select the required files.
   - Add the Selenium-java-4.23.0 JAR file.
4. Create a Java class named GridTest in Eclipse.

Step 6: Write Java Program

--- GridTest.java ---

```
package SeleniumGrid;

import java.net.URL;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.remote.RemoteWebDriver;

public class GridTest {
```

```
    public static void main(String[] args) {
        try {
            ChromeOptions options = new ChromeOptions();

            WebDriver driver = new RemoteWebDriver(
                new URL("http://localhost:4444"),
                options
            );

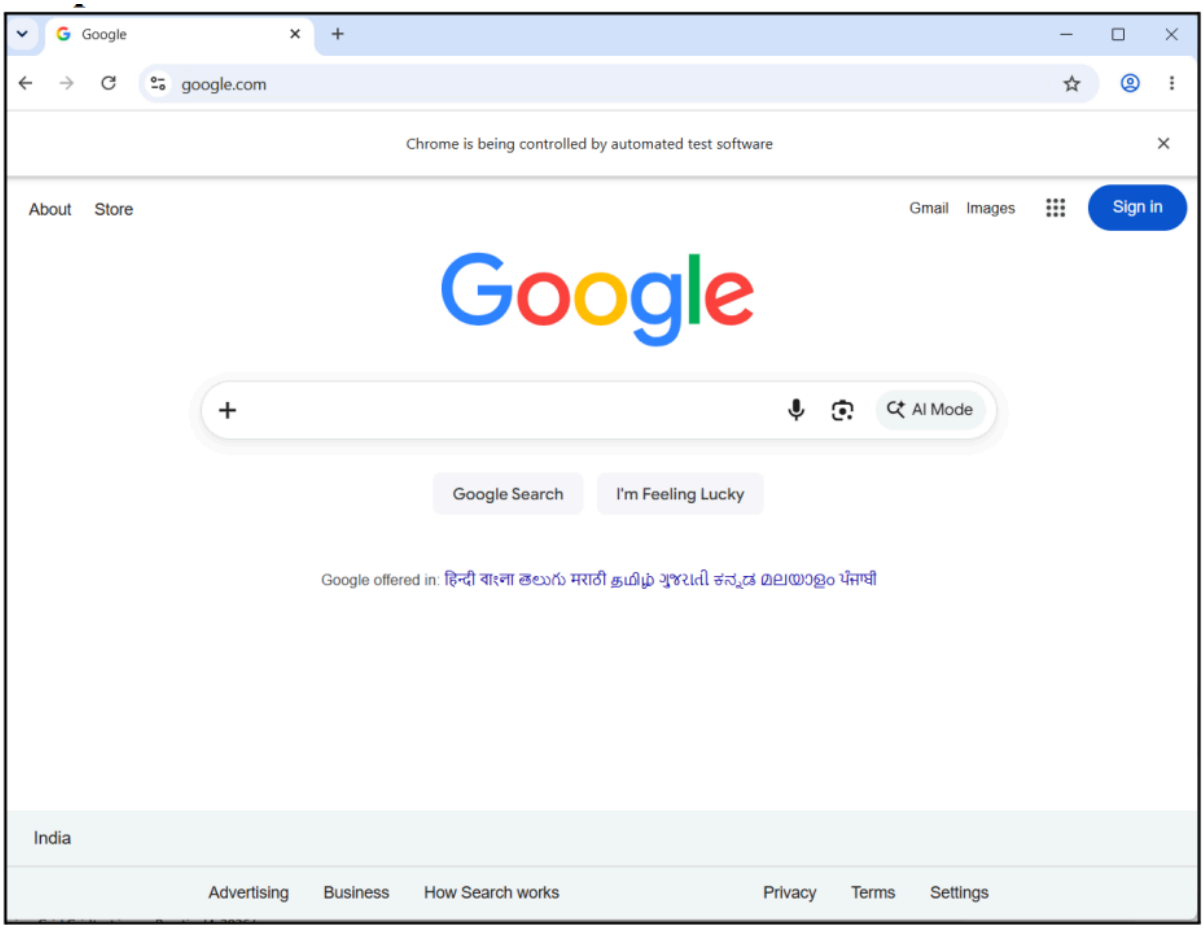
            driver.get("https://www.google.com");
            System.out.println("Page Title : " + driver.getTitle());

            Thread.sleep(3000);
            driver.quit();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Step 7: Execute the Program

1. Run the Java program.
2. The request is sent to the Selenium Hub.
3. The Hub forwards the request to a compatible Chrome Node.
4. The Chrome browser opens automatically.
5. The Google homepage is displayed.
6. The page title is printed on the console.

EXPECTED OUTPUT:
Page Title : Google



Practical 5

Login Automation and Validation

AIM:

Write an automation script to perform login on a specified web page. Verify successful login using assertions such as URL validation, welcome message validation, and logout visibility check.

TOOLS REQUIRED:

1. Selenium WebDriver (Java/PHP)

OBJECTIVES:

1. To automate login functionality.
2. To verify successful authentication.
3. To use assertions for validation.

SOFTWARE REQUIREMENTS:

1. Java JDK
2. Eclipse IDE
3. Selenium WebDriver
4. Chrome Browser

PROCEDURE:

Step 1: Open Eclipse

1. Open Eclipse.
2. Select File > New > Java Project.
3. Enter the project name: LoginAutomation.
4. Click Finish.

Step 2: Add Selenium JARs to the Project

1. Right-click the project.
2. Select Build Path > Configure Build Path.
3. Open Libraries > Classpath.
4. Click Add External JARs and select the required Selenium JAR files.

Step 3: Write the Program in Eclipse

--- LoginTest.java ---

```
package automation;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://practicetestautomation.com/practice-test-login/");

        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();

        String actualURL = driver.getCurrentUrl();
        String expectedURL =
            "https://practicetestautomation.com/logged-in-successfully/";

        if (actualURL.equals(expectedURL)) {
            System.out.println("URL Validation Passed");
        } else {
            System.out.println("URL Validation Failed");
        }

        WebElement message = driver.findElement(By.tagName("h1"));

        if (message.getText().equals("Logged In Successfully")) {
            System.out.println("Welcome Message Verified");
        } else {
            System.out.println("Message Validation Failed");
        }

        WebElement logout = driver.findElement(By.linkText("Log out"));
```

```
        if (logout.isDisplayed()) {
            System.out.println("Logout Button Displayed");
        } else {
            System.out.println("Logout Button Not Found");
        }

        driver.quit();
    }
}
```

Step 4: Run the Program

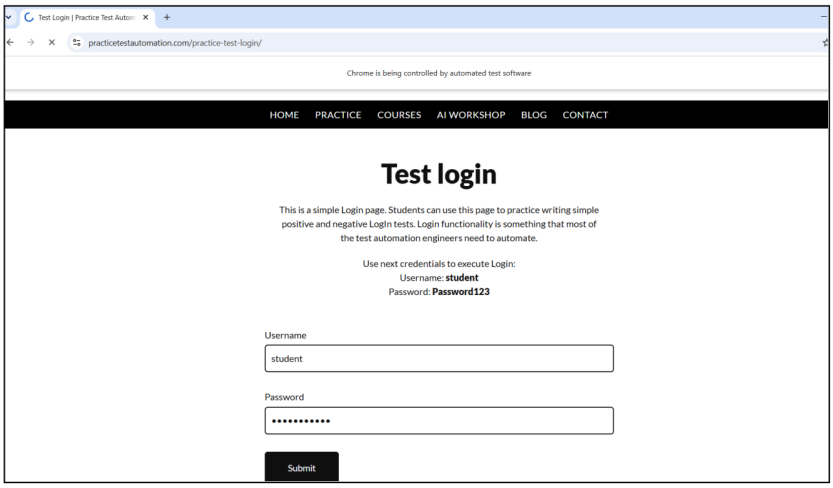
1. Right-click LoginTest.java.
2. Choose Run As > Java Application.
3. Chrome Browser opens automatically.
4. The browser performs the following actions:
   - Opens the website.
   - Enters the username.
   - Enters the password.

   - Clicks Login.
   - Verifies the login result.
   - Closes the browser.

Step 5: Observe Console Output

URL Validation Passed
Welcome Message Verified
Logout Button Displayed

OUTPUT:


Test Login | Practice Test Autom
practicetestautomation.com/practice-test-login/
Chrome is being controlled by automated test software
HOME PRACTICE COURSES AI WORKSHOP BLOG CONTACT
Test login
This is a simple Login page. Students can use this page to practice writing simple positive and negative LogIn tests. Login functionality is something that most of the test automation engineers need to automate.
Use next credentials to execute Login:
Username: student
Password: Password123
Username
student
Password
Submit

Practical 6

Web Element Interaction and Synchronization Handling

Aim

Develop a script to automate interaction with various web elements (textbox, radio button, checkbox, dropdown, alert, and frames). Implement implicit and explicit waits for synchronization handling.

Tool Required

Java JDK 17 or above
Eclipse IDE
Selenium WebDriver
Chrome Browser

Objectives

To identify and interact with web elements.
To handle synchronization issues.
To implement implicit and explicit waits.

Theory

Selenium WebDriver is an open-source automation testing tool used to automate web browsers. It enables testers to interact with web elements such as textboxes, buttons, radio buttons, checkboxes, dropdown lists, alerts, and frames. Modern web applications load content dynamically. Sometimes Selenium tries to interact with an element before it becomes available, resulting in errors such as NoSuchElementException or ElementNotInteractableException. To avoid such synchronization issues, Selenium provides Implicit Wait and Explicit Wait, allowing the script to wait until the required elements become available.

Website Used

DemoQA
https://demoqa.com

Algorithm

Launch the Chrome browser.
Maximize the browser window.
Apply an Implicit Wait.
Open the DemoQA website.
Enter text into textboxes.

Select a radio button.
Select a value from the dropdown list.
Handle the alert box.
Switch to the frame and read the displayed text.
Apply Explicit Wait for a dynamically loaded element.
Close the browser.
"Chrome, Wait, Open, Find, Action, Select, Alert, Frame, Wait, Quit.

Program

```
package practical6;

import java.time.Duration;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;

public class Practical6 {
    public static void main(String[] args) {
        WebDriver d = new ChromeDriver();
        d.manage().window().maximize();
        // Implicit Wait
        d.manage().timeouts().implicitlyWait(Duration.ofSeconds(5));
        // Explicit Wait
        WebDriverWait wait =
                new WebDriverWait(d, Duration.ofSeconds(10));
        // 1. Textbox
        d.get("https://demoqa.com/text-box");
        d.findElement(By.id("userName")).sendKeys("Rahul");
        // 2. Radio Button
        d.get("https://demoqa.com/radio-button");
        d.findElement(By.id("yesRadio")).click();
        // 3. Dropdown
        d.get("https://demoqa.com/select-menu");
        Select s = new Select(d.findElement(By.id("oldSelectMenu")));
```

```
        s.selectByVisibleText("Purple");
        // 4. Alert
        d.get("https://demoqa.com/alerts");
        d.findElement(By.id("alertButton")).click();
        d.switchTo().alert().accept();

        // 5. Frame
        d.get("https://demoqa.com/frames");
        d.switchTo().frame("frame1");

        System.out.println(
                d.findElement(By.id("sampleHeading")).getText()
        );

        d.switchTo().defaultContent();

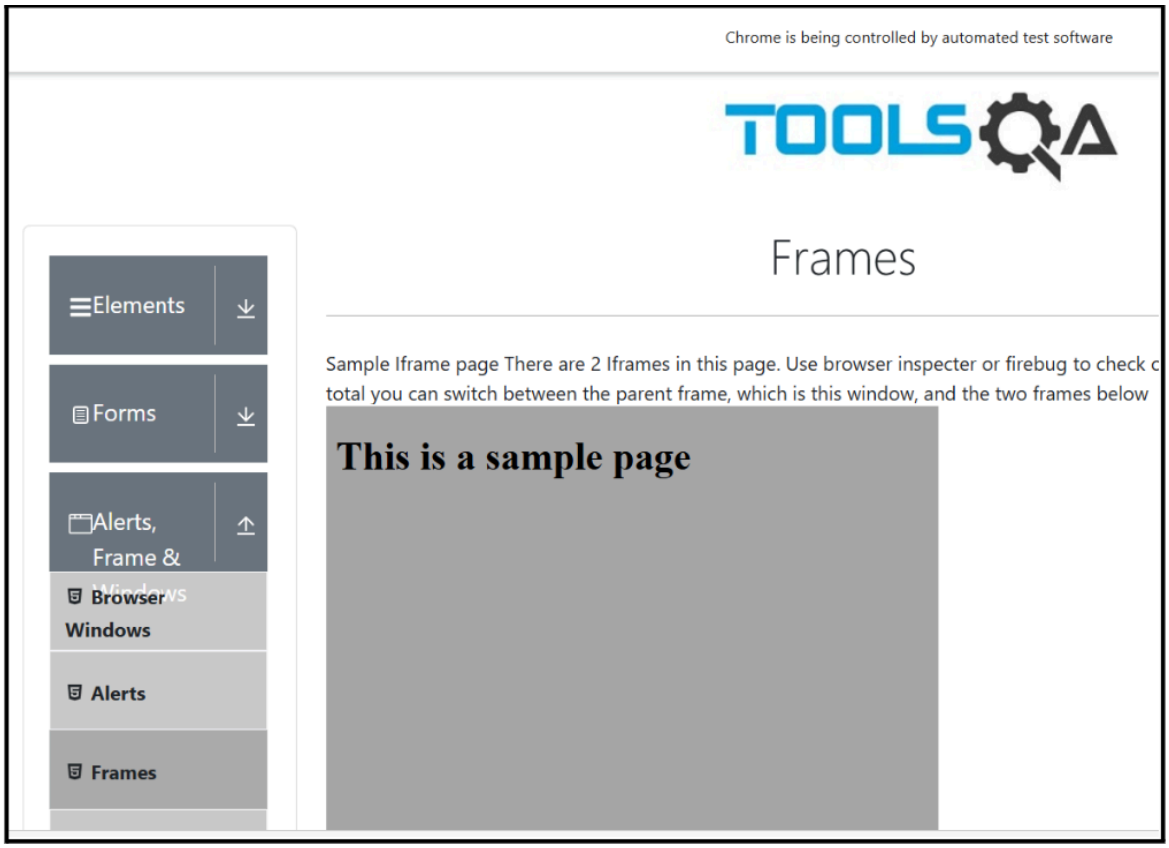
        // 6. Explicit Wait
        d.get("https://demoqa.com/dynamic-properties");
        wait.until(
                ExpectedConditions.elementToBeClickable(
                        By.id("visibleAfter")
                )
        ).click();

        d.quit();
    }
}
```

Result

The Selenium WebDriver script successfully interacted with various web elements such as textboxes, radio buttons, dropdown lists, alerts, and frames. Synchronization was achieved using
both Implicit Wait and Explicit Wait, and the automation script executed successfully.

OUTPUT:


Chrome is being controlled by automated test software
TOOLSQA
Frames
Elements
Forms
Alerts, Frame &
Browser Windows
Alerts
Frames
Sample Iframe page There are 2 Iframes in this page. Use browser inspecter or firebug to check c
total you can switch between the parent frame, which is this window, and the two frames below
This is a sample page

Practical 7

Data-Driven Testing Using Excel Integration

AIM:

Write a program to update 10 student records in an Excel file and validate changes on the web application. Perform data reading, writing, and verification using Apache POI library

PRE-REQUISITES:

1. Eclipse IDE.
2. JXL.JAR library.

PROCEDURE:

1. Open Eclipse and select your workspace directory. Click Launch.
2. Create a Java Project by selecting File > New > Java Project.
3. Name the project “p5”, click Finish, and select “Don't Create module”.
4. Close the Welcome tab.
5. Create a package by right-clicking the project name and selecting New > Package. Enter a package name and click Finish.
6. Create a class by right-clicking the project name and selecting New > Class. Name it Excelwriter.
7. Add the JXL JAR file:
   - Right-click the project name and select Build Path > Configure Build Path.
   - Open Java Build Path > Libraries > Classpath.
   - Click Add External JARs.
   - Select the downloaded jxl.jar file and click Apply and Close.
8. Create the Java program below in Excelwriter.java.

PROGRAM: Excelwriter.java

```
package excelwrite;

import jxl.*;
import jxl.write.*;
import jxl.write.Number;
import java.io.*;
import java.util.Locale;

public class Excelwriter {
   public static void main(String[] args) throws IOException, WriteException {
      int r = 0, c = 0;
```

```
String header[] = {
   "Student Name", "Subject1", "Subject2", "Subject3", "Total"
};

String sname[] = {
   "Carls", "James", "Paul", "Philip", "Smith",
   "Thomson", "Rhodey", "Stark", "Gary", "AnneMarie"
};

int marks[] = {50, 45, 60, 55, 70, 45, 67, 78, 89, 90};

File file = new File("student.xls");
WorkbookSettings wbSettings = new WorkbookSettings();
wbSettings.setLocale(new Locale("en", "EN"));

WritableWorkbook workbook =
   Workbook.createWorkbook(file, wbSettings);

workbook.createSheet("Report", 0);
WritableSheet excelSheet = workbook.getSheet(0);

// Creating the header row
for (r = 0; r < 1; r++) {
   for (c = 0; c < header.length; c++) {
      Label l = new Label(c, r, header[c]);
      excelSheet.addCell(l);
   }
}

// Filling names in column 1
for (r = 1; r <= sname.length; r++) {
   for (c = 0; c < 1; c++) {
      Label l = new Label(c, r, sname[r - 1]);
      excelSheet.addCell(l);
   }
}

// Filling marks in columns 2, 3 and 4
```

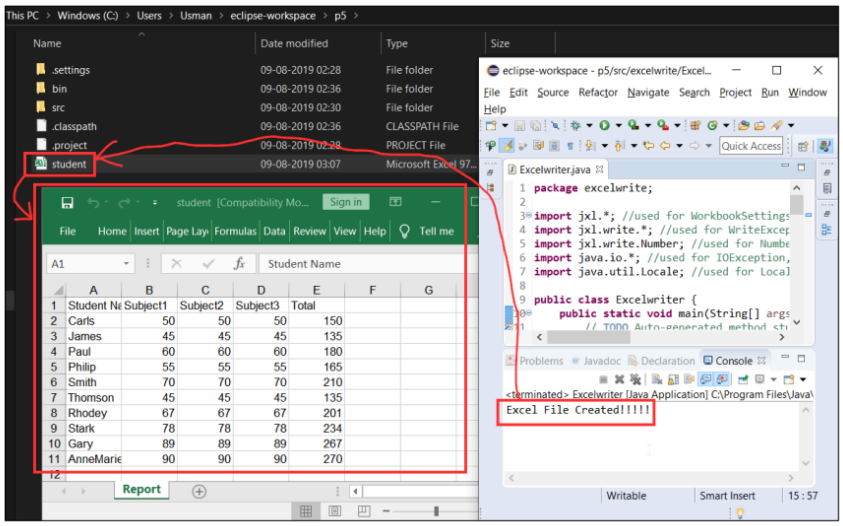
```
        for (r = 1; r <= sname.length; r++) {
            for (c = 1; c < 4; c++) {
                Number num = new Number(c, r, marks[r - 1]);
                excelSheet.addCell(num);
            }
        }

        // Filling the total column
        for (r = 1; r <= sname.length; r++) {
            for (c = 4; c < 5; c++) {
                int total = marks[r - 1] + marks[r - 1] + marks[r - 1];
                Number num = new Number(c, r, total);
                excelSheet.addCell(num);
            }
        }

        workbook.write();
        workbook.close();

        System.out.println("Excel File Created!!!!!");
    }
}
```

9. Run Excelwriter.java from Eclipse.
EXPECTED OUTPUT:



Practical 8

Data Extraction and Score Analysis from Web Table

AIM:

Write and test a program to select the number of students from an Excel file (table) who have scored 60 or more in any one subject (or all subjects).

PRE-REQUISITES:

1. Eclipse IDE.
2. JXL.JAR library.
3. The Excel file student.xls, which contains the student marks.

PROCEDURE:

1. Open Eclipse and select your workspace directory. Click Launch.
2. Create a Java Project by selecting File > New > Java Project.
3. Name the project “p6”, click Finish, and select “Don't Create module”.
4. Close the Welcome tab.
5. Create a package by right-clicking the project name and selecting New > Package. Enter a package name and click Finish.
6. Create a class by right-clicking the project name and selecting New > Class. Name it Excelreader.
7. Add the JXL JAR file:
   - Right-click the project name and select Build Path > Configure Build Path.
   - Open Java Build Path > Libraries > Classpath.
   - Click Add External JARs, select jxl.jar, then click Apply and Close.
8. Create the Java program below in Excelreader.java.
9. Update the file path in setInputFile() to the actual location of student.xls on your computer.
10. Run the program from Eclipse.

PROGRAM: Excelreader.java

```
package excelread;

import java.io.File;
import java.io.IOException;
import jxl.Cell;
import jxl.CellType;
import jxl.Sheet;
import jxl.Workbook;
```

```
import jxl.read.biff.BiffException;

public class Excelreader {
   private String inputFile;

   public void setInputFile(String inputFile) {
      this.inputFile = inputFile;
   }

   public void read() throws IOException {
      File inputWorkbook = new File(inputFile);
      Workbook w;
      boolean flag = false;
      int count = 0;

      try {
         w = Workbook.getWorkbook(inputWorkbook);

         // Get the first sheet
         Sheet sheet = w.getSheet(0);

         // Check numeric cells, excluding the last column
         for (int j = 0; j < sheet.getRows(); j++) {
            for (int i = 0; i < sheet.getColumns() - 1; i++) {
               Cell cell = sheet.getCell(i, j);

               if (cell.getType() == CellType.NUMBER) {
                  if (Integer.parseInt(cell.getContents()) >= 60) {
                     flag = true;

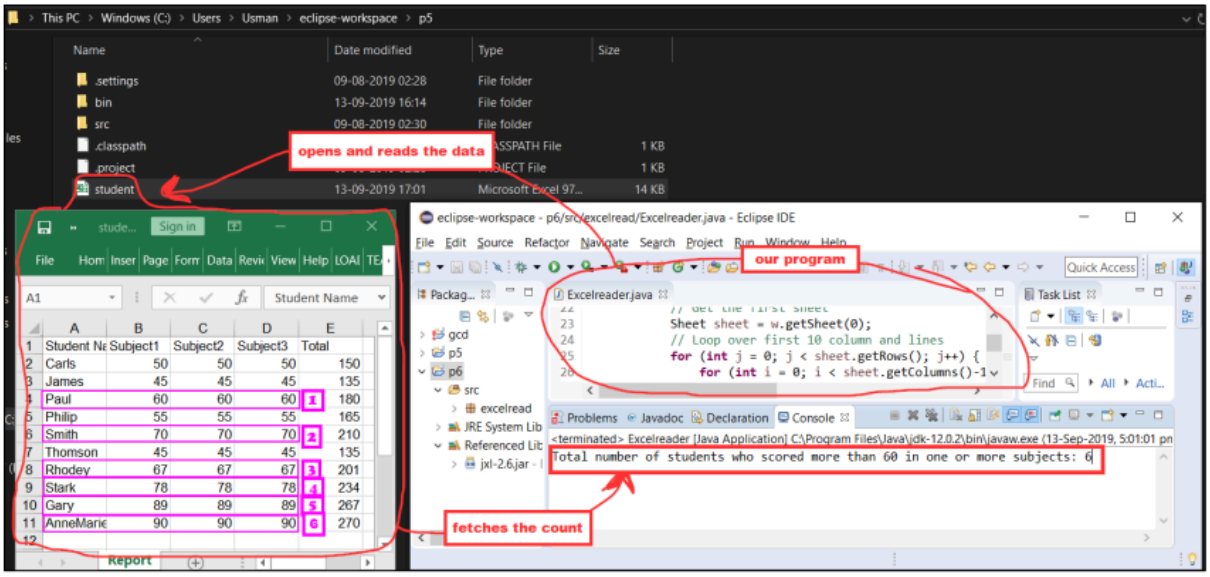
                     if (flag == true) {
                        count++;
                        flag = false;
                     }
                     break;
                  }
               }
            }
         }
```

```
            System.out.println(
               "Total number of students who scored more than 60 in one or more subjects: "
               + count
            );

            w.close();
         } catch (BiffException e) {
            e.printStackTrace();
         }
      }

      public static void main(String[] args) throws IOException {
         Excelreader test = new Excelreader();
         test.setInputFile("C:\\Users\\YourName\\eclipse-workspace\\p5\\student.xls");
         test.read();
      }
}
```

EXPECTED OUTPUT:



Practical 9

Object Identification and Counting on Web Page

AIM

Write a program to identify and count the total number of objects present on a web page including links, images, buttons, and input elements. Display categorized counts.

PRE-REQUISITES

1. JDK installed.
2. Eclipse IDE installed.
3. Selenium Server Driver and Client Driver (JAR files) available.
4. GeckoDriver available.
5. A stable Internet connection.

PROCEDURE

1. Open Eclipse and select your workspace.
2. Create a Java Project (File > New > Java Project). Name it "p8".
3. Finish the project setup. If prompted, choose "Don't Create module".
4. Create a package in the project.
5. Create a Java class named FindAllLinks.
6. Add the Selenium Server and Client JAR files:
   - Right-click the project > Build Path > Configure Build Path.
   - Open Java Build Path > Libraries > Classpath.
   - Select Add External JARs, choose the Selenium JAR files, then click Apply and Close.
7. Update the GeckoDriver path in the code below to match its location on your computer.
8. Run the Java file from Eclipse.

PROGRAM: FindAllLinks.java

```
package p8;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class FindAllLinks {
```

```
    static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";

    public static void main(String[] args) {
        System.setProperty("webdriver.gecko.driver", driverPath);

        WebDriver driver = new FirefoxDriver();
        String appUrl = "https://www.google.co.in/";
        driver.get(appUrl);

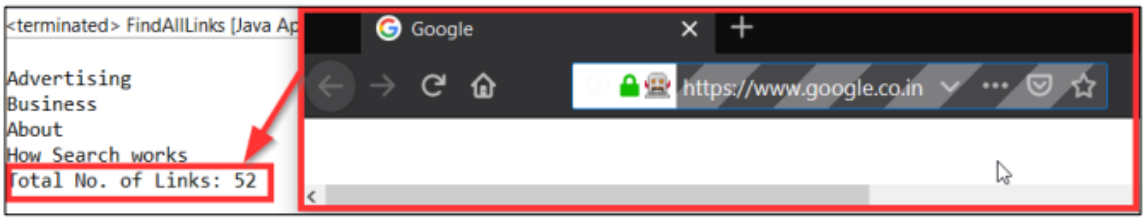
        java.util.List<WebElement> links =
                driver.findElements(By.tagName("a"));

        for (int i = 1; i < links.size(); i = i + 1) {
            System.out.println(links.get(i).getText());
        }

        System.out.println("Total No. of Links: " + links.size());

        // driver.quit();
    }
}
```

EXPECTED RESULT



Practical 10

List and Combo Box Item Verification

AIM

Automate the process of identifying a list or dropdown box on a web page and determine the total number of items. Validate default selected value and perform selection testing.

PRE-REQUISITES

1. JDK installed.
2. Eclipse IDE installed.
3. Selenium Server Driver and Client Driver (JAR files) available.
4. GeckoDriver available.
5. A stable Internet connection.

PROCEDURE

1. Open Eclipse and select your workspace.
2. Create a Java Project named "p9". Finish the project setup; if prompted, choose "Don't Create module".
3. Create a package and a Java class in the project.
4. Add the Selenium Server and Client JAR files:
   - Right-click the project > Build Path > Configure Build Path.
   - Open Java Build Path > Libraries > Classpath.
   - Select Add External JARs, choose the Selenium JAR files, then click Apply and Close.
5. Optionally create a local HTML file named combobox.html with a dropdown list, then open it in a browser to obtain its file URL.

OPTIONAL LOCAL HTML FILE: combobox.html

```
<select id="continents">
    <option value="Asia">Asia</option>
    <option value="Europe">Europe</option>
    <option value="Africa">Africa</option>
</select>
```

PROGRAM: ComboBox.java

```
package p9;

import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.support.ui.Select;

public class ComboBox {
    static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";

    public static void main(String[] args) {
        System.setProperty("webdriver.gecko.driver", driverPath);

        WebDriver driver = new FirefoxDriver();

        // Dynamic URL (website) shown in the original journal:
        String appUrl = "https://www.toolsqa.com/automation-practice-form/";

        // To use the local HTML file instead, comment out the line above
        // and uncomment the next line after updating the path:
        // String appUrl = "file:///D:/path/to/combobox.html";

        driver.get(appUrl);

        Select oSelect = new Select(driver.findElement(By.id("continents")));
        // The journal also shows this alternative locator:
        // Select oSelect = new Select(driver.findElement(By.tagName("select")));

        List<WebElement> oSize = oSelect.getOptions();
        int iListSize = oSize.size();

        for (int i = 0; i < iListSize; i++) {
            // Store the text of the option
            String sValue = oSelect.getOptions().get(i).getText();

            // Print the stored option text
            System.out.println(sValue);
```

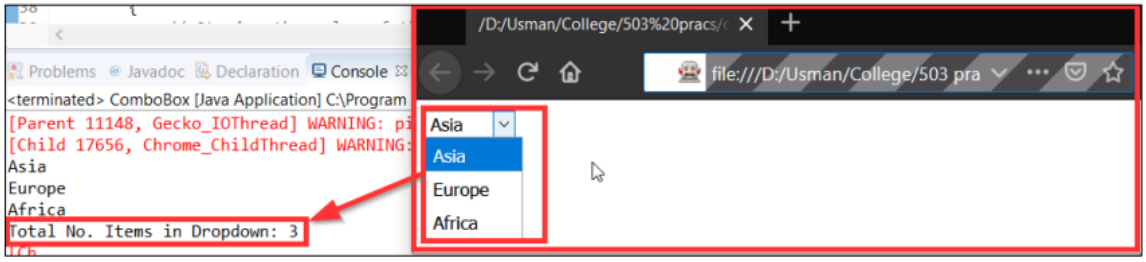
```
        }

        System.out.println("Total No. Items in Dropdown: " + iListSize);

        // Close the browser when finished:
        driver.quit();
    }
}
```

EXPECTED RESULT

```
Asia
Europe
Africa
Total No. Items in Dropdown: 3
```



Practical 11

Checkbox Identification and Validation

AIM

Write and test a program to count the number of checkboxes on a page and find the checked and unchecked counts.

PRE-REQUISITES

1. JDK installed.
2. Eclipse IDE installed.
3. Selenium Server Driver and Client Driver (JAR files) available.
4. GeckoDriver available.
5. A stable Internet connection.

PROCEDURE

1. Open Eclipse and select your workspace.
2. Create a Java Project named "p10". Finish the project setup; if prompted, choose "Don't Create module".
3. Create a package and a Java class named MultiCheckBox.
4. Add the Selenium Server and Client JAR files:
   - Right-click the project > Build Path > Configure Build Path.
   - Open Java Build Path > Libraries > Classpath.
   - Select Add External JARs, choose the Selenium JAR files, then click Apply and Close.
5. Optionally create a local HTML file named multicheckbox.html using the markup below.
6. Update the GeckoDriver path and select the URL you want to test.
7. Run the Java file from Eclipse.

LOCAL HTML FILE: multicheckbox.html

```
<!DOCTYPE html>
<html>
<body>
  <input type="checkbox" value="A">A<br>
  <input type="checkbox" value="B" checked>B<br>
  <input type="checkbox" value="C">C<br>
  <input type="checkbox" value="D" checked>D<br>
  <input type="checkbox" value="E">E<br>
</body>
</html>
```

PROGRAM: MultiCheckBox.java

```
package p10;

import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class MultiCheckBox {
   static String driverPath = "C:\\Users\\Usman\\503\\geckodriver.exe";

   public static void main(String[] args) {
      System.setProperty("webdriver.gecko.driver", driverPath);

      WebDriver driver = new FirefoxDriver();

      // Dynamic URL (website) from the original journal:
      String appUrl = "https://www.toolsqa.com/automation-practice-form/";

      // To use the local HTML file instead, comment the line above and
      // uncomment the next line after updating the path:
      // String appUrl = "file:///D:/path/to/multicheckbox.html";

      driver.get(appUrl);

      List<WebElement> checkBoxes =
            driver.findElements(By.xpath("//input[@type='checkbox']"));

      int checkedCount = 0;
      int uncheckedCount = 0;

      for (int i = 0; i < checkBoxes.size(); i++) {
         System.out.println(i + " checkbox is selected "
               + checkBoxes.get(i).isSelected());

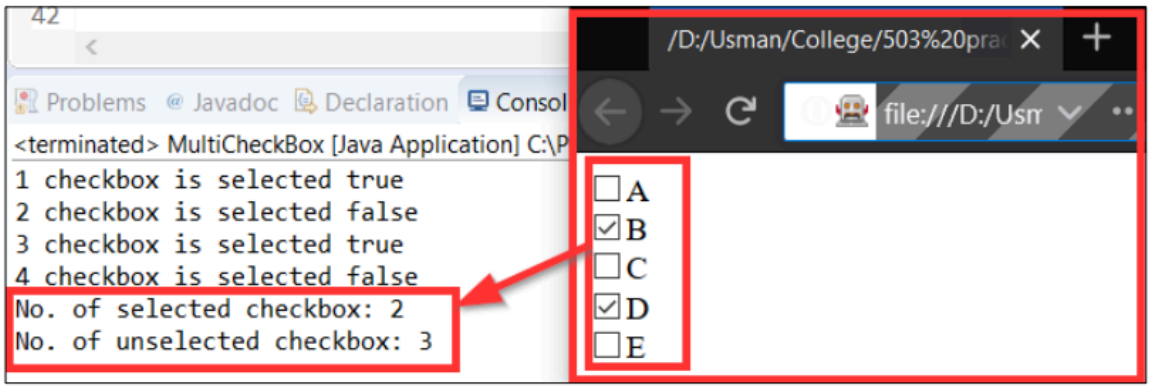
         if (checkBoxes.get(i).isSelected()) {
```

```
                checkedCount++;
            } else {
                uncheckedCount++;
            }
        }

        System.out.println("No. of selected checkbox: " + checkedCount);
        System.out.println("No. of unselected checkbox: " + uncheckedCount);

        driver.quit();
    }
}
```

EXPECTED RESULT



Practical 12

Dynamic Web Table Handling and Sorting Validation

Aim

To extract data from a dynamic web table, validate the sorting functionality, and verify the correctness of filtered results.

Tool Required:

Selenium WebDriver

Objectives:

- To identify and extract data from a web table.
- To handle dynamic web table elements.
- To perform sorting operations.
- To validate sorted data.
- To verify filtered results.

Theory

A dynamic web table displays data in rows and columns, where the data can change when sorting or filtering is performed.
Selenium WebDriver can be used to read table data and perform actions such as clicking Sort and Filter buttons.

PROCEDURE

Step 1: Create HTML Page — table.html

D:\TYCS STQA\table.html

Use the following complete HTML. The Sort Marks button sorts the student rows by marks in descending order.

```
<!DOCTYPE html>
<html>
<head>
   <meta charset="UTF-8">
   <title>Student Table</title>
</head>
```

```
<body>
  <h2>Student Table</h2>

  <button id="sort" onclick="sortTable()">Sort Marks</button>

  <table id="students" border="1">
    <thead>
      <tr>
        <th>Name</th>
        <th>Marks</th>
      </tr>
    </thead>
    <tbody>
      <tr><td>Rahul</td><td>75</td></tr>
      <tr><td>Priya</td><td>50</td></tr>
      <tr><td>Amit</td><td>85</td></tr>
    </tbody>
  </table>

  <script>
    function sortTable() {
      const table = document.getElementById("students");
      const tbody = table.tBodies[0];
      const rows = Array.from(tbody.rows);

      rows.sort((a, b) =>
        Number(b.cells[1].textContent) -
        Number(a.cells[1].textContent)
      );

      rows.forEach(row => tbody.appendChild(row));
    }
  </script>
</body>
</html>
```

Step 2: Open Eclipse

1. Open Eclipse 2023.
2. Create a new Java Project.
3. Create a package named practical2.
4. Create a Java class named Practical2.

Step 3: Add Selenium Library

Right-click the project and select:

Build Path → Configure Build Path → Libraries → Classpath → Add External JARs.

Add the required Selenium JAR files.

PROGRAM: Practical2.java

```
package practical2;

import java.util.List;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class Practical2 {
    public static void main(String[] args) {
        WebDriver d = new ChromeDriver();

        d.get("file:///D:/TYCS%20STQA/table.html");

        // Read table rows
        List<WebElement> rows =
                d.findElements(By.xpath("//table[@id='students']/tbody/tr"));

        System.out.println("Student Data:");

        for (int i = 0; i < rows.size(); i++) {
            List<WebElement> cells =
```

```
                rows.get(i).findElements(By.tagName("td"));

            System.out.println(
                cells.get(0).getText() + " - " +
                cells.get(1).getText()
            );
        }

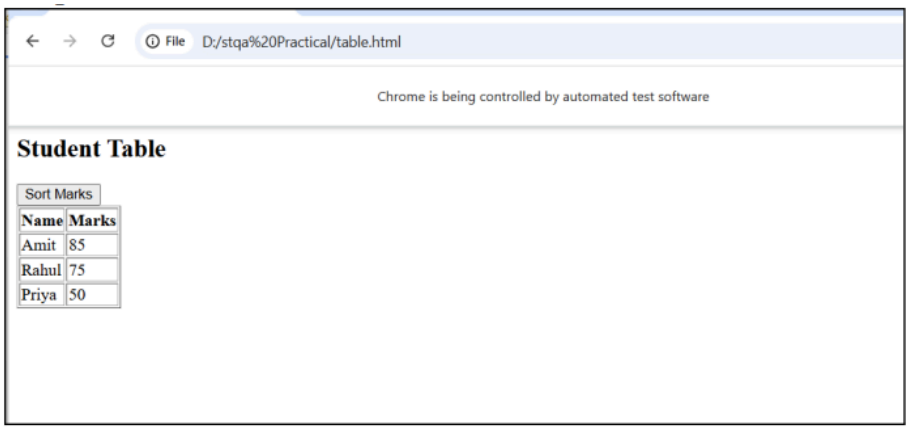
        // Sort table by marks
        d.findElement(By.id("sort")).click();

        // Verify the highest marks after sorting
        String marks = d.findElement(
              By.xpath("//table[@id='students']/tbody/tr[1]/td[2]")
        ).getText();

        System.out.println("Highest Marks = " + marks);

        d.quit();
    }
}
```

EXPECTED OUTPUT



Practical 13

File Upload and Download Automation

Aim

To automate file upload and download operations using Selenium WebDriver and verify the successful upload and downloaded file.

Tool Required:

Selenium WebDriver
Java Robot Class

Objectives:

To automate file upload.
To automate file download.
To verify successful file upload.
To verify the existence of the downloaded file.
To understand file handling in Selenium.

Step 1: Create HTML Page

Create a file named:
file.html
Save it in:
"C:\Users\samit\Desktop\stqa\file.html"

```
<html>
<body>
<h2>File Upload and Download</h2>
<input type="file" id="upload">
<br><br>
<a href="sample.txt" download id="download">
Download File
</a>
</body>
</html>
```

Step 2: Open Eclipse

Open Eclipse 2023
Create a new Java Project

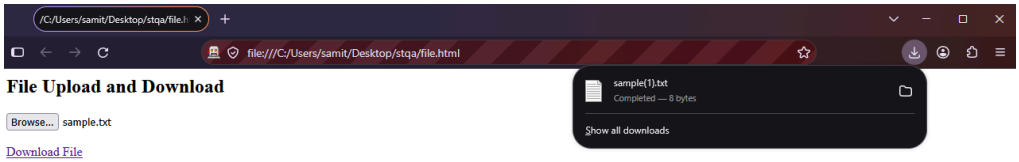
Step 3: Add Selenium Library

Right-click Project → Build Path → Configure Build Path → Libraries → Classpath →

Add External JARs

```
package sam;
import java.io.File;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.firefox.FirefoxDriver;
public class pratical3 {
public static void main(String[] args) {
WebDriver d = new FirefoxDriver();
d.get("file:///C:/Users/samit/Desktop/stqa/file.html");
// Upload file
d.findElement(By.id("upload"))
.sendKeys("D:\\TYCS STQA\\student.txt");
System.out.println("File Uploaded");
// Download file
d.findElement(By.id("download")).click();
// Check downloaded file
File f = new File(
"C:\\Users\\Public\\Downloads\\sample.txt");
if (f.exists())
System.out.println("File Downloaded");
else
System.out.println("Download Failed");
d.quit();
}
}
```

OUTPUT:


/C:/Users/samit/Desktop/stqa/file.h
file:///C:/Users/samit/Desktop/stqa/file.html
File Upload and Download
Browse...
sample.txt
Download File
sample(1).txt
Completed — 8 bytes
Show all downloads

Practical 15

Page Object Model (POM) Framework Implementation

Aim:
To design and implement a Selenium automation framework using the Page Object Model (POM) and create reusable page classes for modular test execution.
Tool Required:
Selenium WebDriver
TestNG

Objectives:
To understand the Page Object Model.
To create reusable page classes.
To separate page elements and test logic.
To implement modular test cases.
To improve code reusability and maintenance

CODE :
LOGINpage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.support.ui.WebDriverWait;
import org.openqa.selenium.support.ui.ExpectedConditions;
import java.time.Duration;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    By username = By.id("edd_user_login");
    By password = By.id("edd_user_pass");
    By loginButton = By.id("edd_login_submit");
```

```
    public LoginPage(WebDriver driver) {
        this.driver = driver;
    }

    public void enterUsername(String user) {
        driver.findElement(username).sendKeys(user);
    }

    public void enterPassword(String pass) {
        driver.findElement(password).sendKeys(pass);
    }

    public void clickLogin() {
        WebDriverWait wait = new WebDriverWait(
                driver, Duration.ofSeconds(10));

        WebElement button = wait.until(
                ExpectedConditions.elementToBeClickable(
                        By.id("edd_login_submit")
                )
        );

        button.click();
    }

    public void login(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
        clickLogin();
    }
}
```

testing.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="POM Test Suite">
    <test name="Login Test">
```

```
        <classes>
            <class name="tests.LoginTest"/>
        </classes>
    </test>
</suite>
```

homepage.java

```
package pages;

import org.openqa.selenium.By;

public class HomePage {

    WebDriver driver;

    By welcomeMessage = By.id("welcome");

    public HomePage(WebDriver driver) {
        this.driver = driver;
    }

    public String getWelcomeMessage() {
        return driver.findElement(welcomeMessage).getText();
    }
}
```

LoginTest.java

```
package tests;

import org.openqa.selenium.WebDriver;

public class LoginTest {

    WebDriver driver;
    LoginPage loginPage;
    HomePage homePage;
```

```
    @BeforeClass
    public void setUp() {

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.get("https://colorlib.com/checkout/my-account/");

        loginPage = new LoginPage(driver);
        homePage = new HomePage(driver);
    }

    @Test
    public void validLoginTest() {

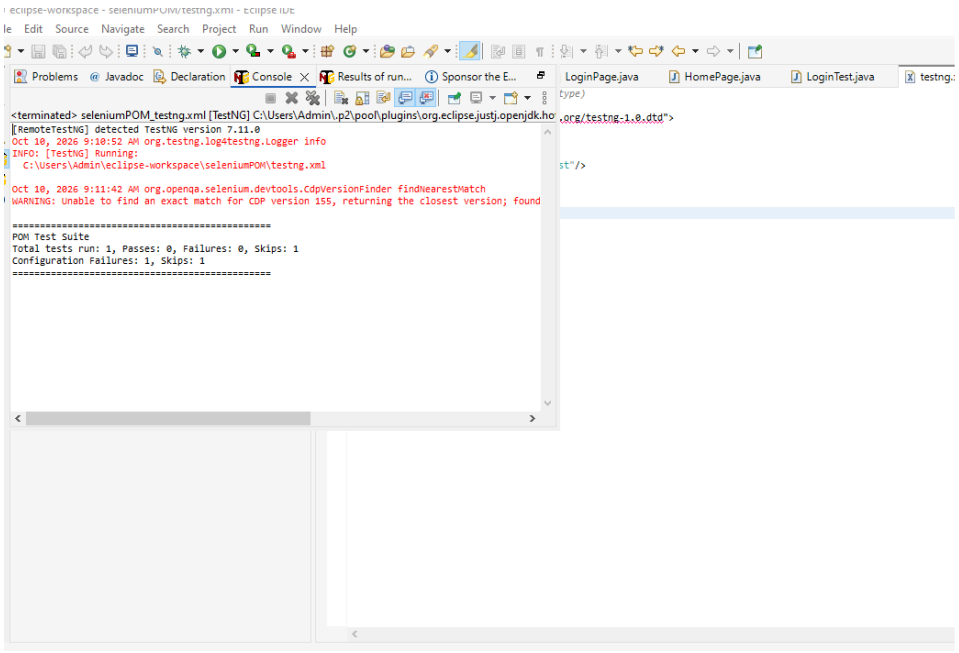
        loginPage.login("testuser", "password");

        String message = homePage.getWelcomeMessage();

        Assert.assertEquals(message, "Welcome");
    }

    @AfterClass
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

OUPUT:


Edit Source Navigate Search Project Run Window Help
Problems Javadoc Declaration Console Results of run... Sponsor the E...
LoginPage.java HomePage.java LoginTest.java
<terminated> seleniumPOM_testng.xml [TestNG] C:\Users\Admin\.p2\pool\plugins\org.eclipse.justj.openjdk.ho
[RemoteTestNG] detected TestNG version 7.11.0
Oct 10, 2026 9:10:52 AM org.testng.log4testng.Logger info
INFO: [TestNG] Running:
C:\Users\Admin\eclipse-workspace\seleniumPOM\testng.xml
Oct 10, 2026 9:11:42 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 155, returning the closest version; found
POM Test Suite
Total tests run: 1, Passes: 0, Failures: 0, Skips: 1
Configuration Failures: 1, Skips: 1

Practical 16

TestNG Framework Implementation and Reporting

Aim:

To create and execute automated test cases using TestNG annotations, priorities, and groups, and generate test execution reports.
Tool Required:
Selenium WebDriver
TestNG
Objectives:

To understand the TestNG framework.
To use TestNG annotations.
To define test case priorities.
To group test cases.
To generate and analyze test reports.

Code:

```
package TestNGDemo;

import org.testng.annotations.AfterClass;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;

public class LoginTest {

    @BeforeClass
    public void setUp() {
        System.out.println("Opening application...");
    }

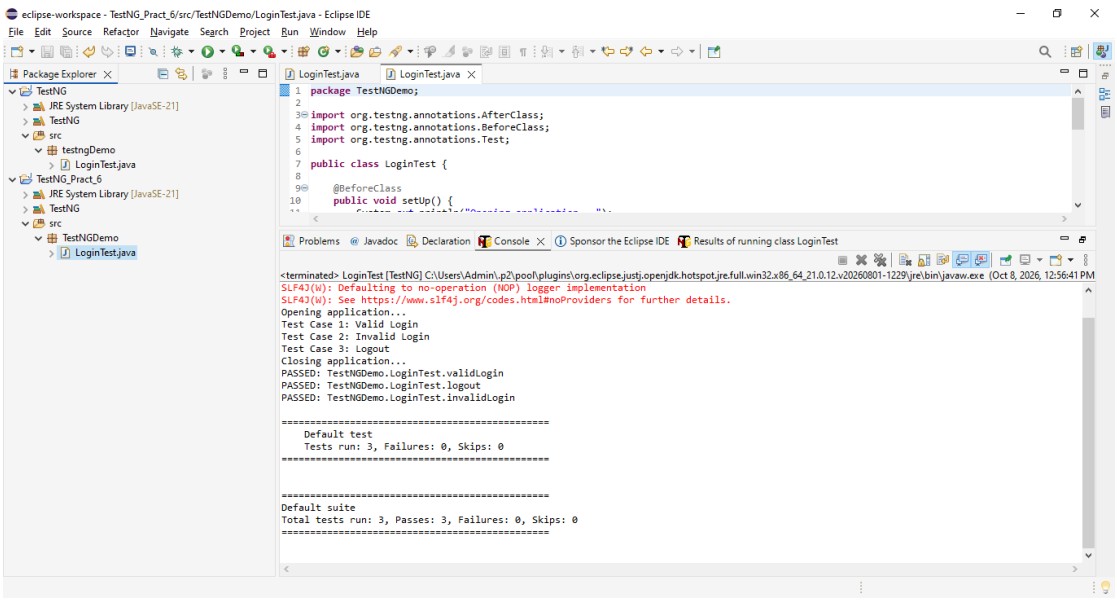
    @Test(priority = 1, groups = {"smoke"})
    public void validLogin() {
        System.out.println("Test Case 1: Valid Login");
    }
```

```
    @Test(priority = 2, groups = {"regression"})
    public void invalidLogin() {
        System.out.println("Test Case 2: Invalid Login");
    }

    @Test(priority = 3, groups = {"smoke", "regression"})
    public void logout() {
        System.out.println("Test Case 3: Logout");
    }

    @AfterClass
    public void tearDown() {
        System.out.println("Closing application...");
    }
}
```

OUTPUT:



Practical 19

Load Testing Using Apache JMeter

AIM

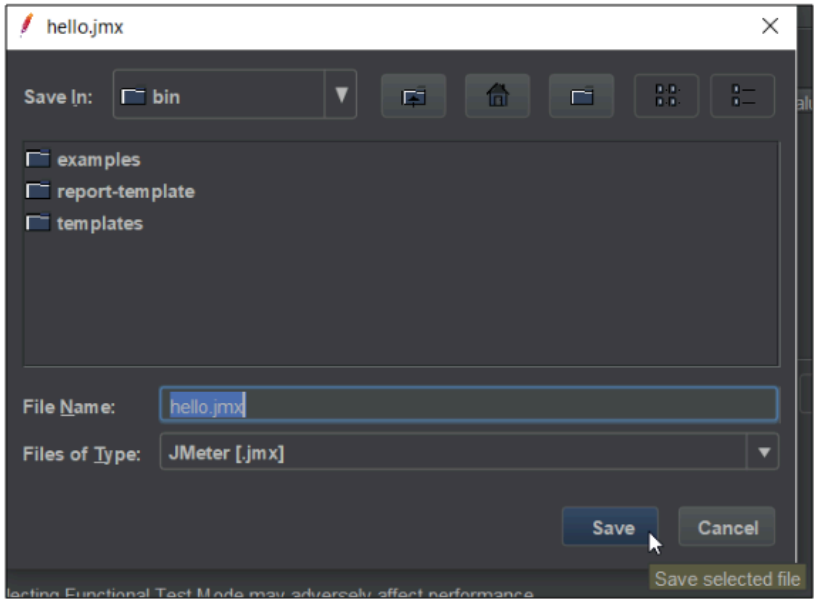
To design and execute a load testing scenario for a web application using Apache JMeter and analyze response time, throughput, and error percentage.

PRE-REQUISITES

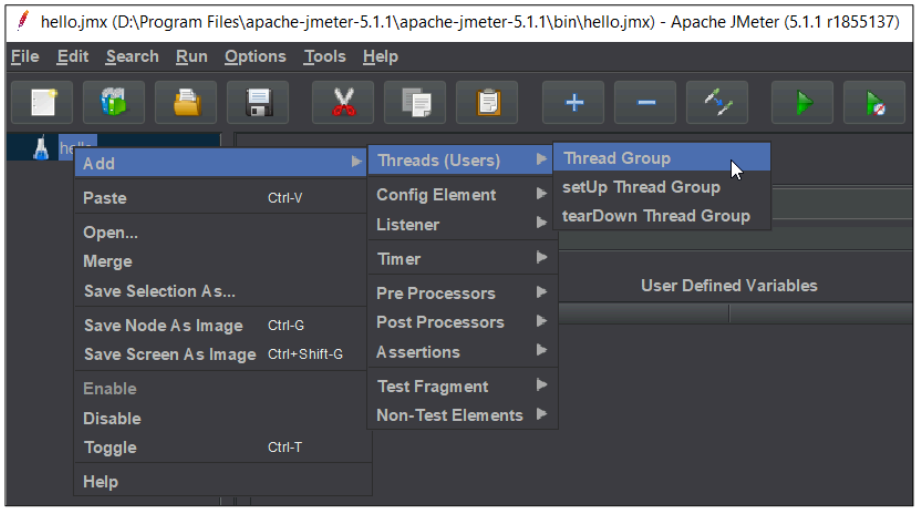
1. JDK:
   - The original journal instructs users to download and install JDK 12.0.2 for Windows 64-bit.
2. Apache JMeter:
   - Visit https://jmeter.apache.org/download_jmeter.cgi
   - The journal specifies Apache JMeter 5.1.1 (requires Java 8+) and the binary archive apache-jmeter-5.1.1.zip.
   - Extract the ZIP file.
   - Navigate to apache-jmeter-5.1.1 > bin. The journal identifies ApacheJMeter.jar as the working space.

PROCEDURE

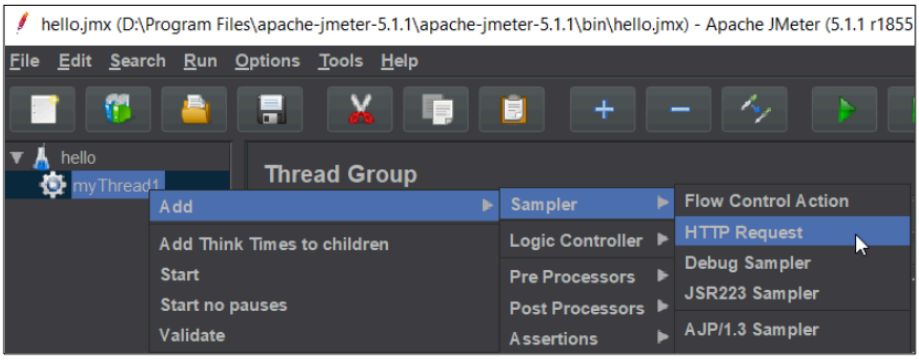
1. Open ApacheJMeter.jar.
2. Name the Test Plan "hello".
3. Save the Test Plan using Ctrl+S. It is saved as a .jmx file.



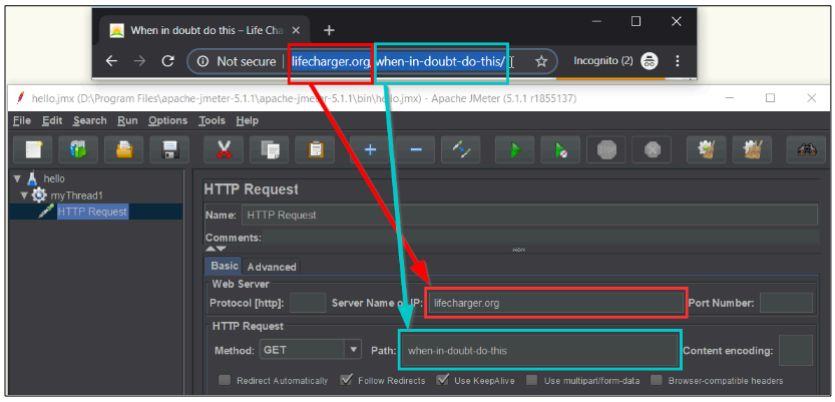
4. Create a Thread Group:
   - Right-click "hello (Test Plan)".
   - Select Add > Thread (Users) > Thread Group.
   - Name the Thread Group "myThread1".
   - Save the test plan.



5. Add an HTTP Request Sampler:
   - Right-click "myThread1 (Thread Group)".
   - Select Add > Sampler > HTTP Request.

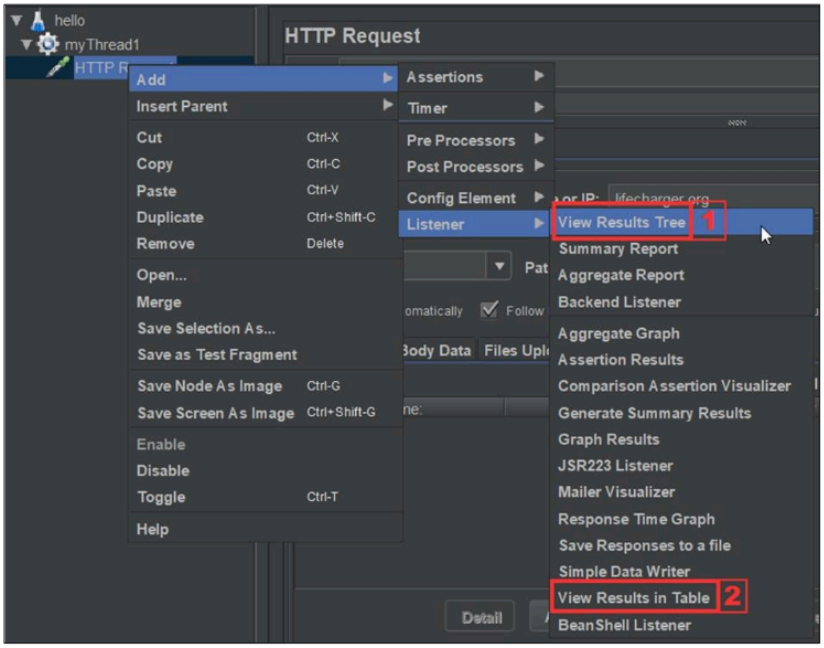


6. Configure the HTTP Request:
   - Visit the website/page you want to test.
   - Enter the website host in "Server Name or IP".
   - Enter the remaining URL path in "Path".

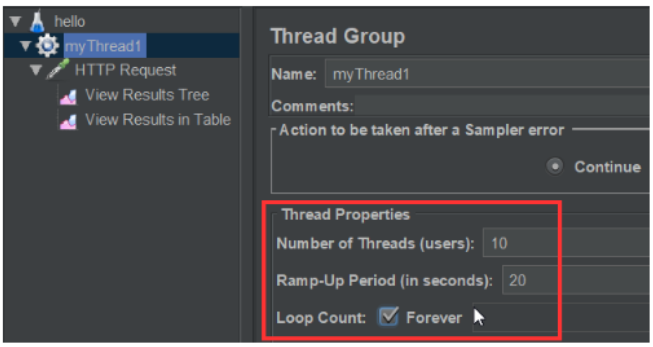


7. Add two listeners under the HTTP Request Sampler:
   - View Results Tree
   - View Results in Table

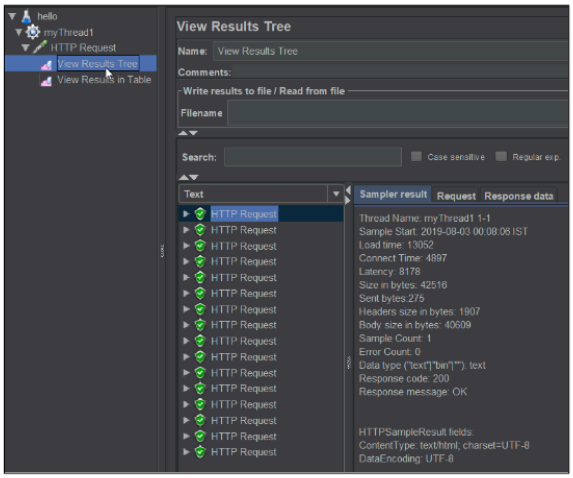
   To add each listener, right-click the HTTP Request Sampler and select Add > Listener, then choose the listener.

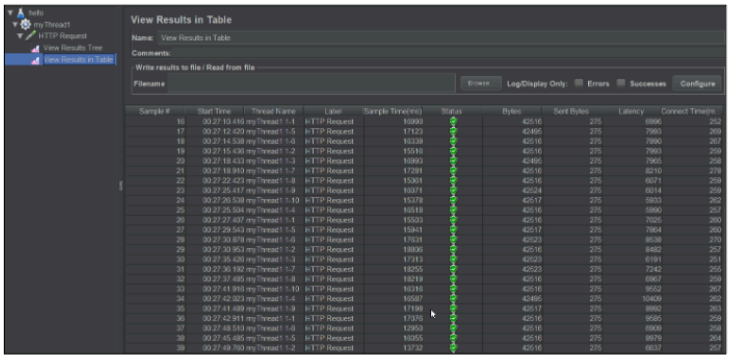


8. Save the test plan using Ctrl+S.



9. Run the test using Ctrl+R and wait for the results.





OBSERVATION / OUTPUT

JMeter displays request results in the View Results Tree and View Results in Table listeners. Review the results to observe the responses from the server.